

COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS | Covering Process Scheduling, Memory Management & Disk Scheduling

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi
Assignment Title	Design and Implementation of OSLearn – An Interactive OS Algorithm Trainer using Streamlit
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Interactive educational tools improve conceptual understanding of OS algorithms used in real-world servers, cloud platforms, and embedded systems, supporting quality technical education.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an interactive Operating Systems simulator that applies core CSA04 concepts — process scheduling (Units I–II), memory management with paging and page replacement (Unit III), and disk scheduling (Unit IV) — to a unified educational training platform called **OSLearn**. Students must integrate multiple algorithm families, produce clear comparative visualizations and step-by-step traces, deploy the system as a lightweight web application using Streamlit, validate correctness and usability with sample test cases, and address basic responsible-computing considerations such as transparency of intermediate steps and reproducibility of results.

C. Problem Statement

Scenario: “OSLearn” – Interactive OS Algorithm Trainer

Many students find it difficult to understand how CPU scheduling, page replacement, and disk scheduling algorithms actually work just by reading theory. Your task is to design and implement an interactive educational web application called **OSLearn** that helps students practice, visualize, and compare these core Operating Systems algorithms through a unified interface.

The system shall integrate three major OS algorithm families:

1. Process Scheduling

Implement FCFS, SJF (Non-preemptive), and Round Robin.

- Users should be able to enter process details (Process ID, Arrival Time, Burst Time, and Time Quantum for Round Robin).

- The system must display a Gantt chart / timeline visualization together with a table showing Waiting Time, Turnaround Time, and Average Waiting/Turnaround Time for each algorithm, enabling side-by-side comparison.

2. Memory Management (Page Replacement)

Implement FIFO and LRU page-replacement algorithms.

- Users will enter the number of frames and a page reference string.
- The system should show the step-by-step frame status after each reference, highlight page faults, and report the total number of page faults for each algorithm so that students can compare performance.

3. Disk Scheduling

Implement FCFS, SSTF, and SCAN.

- Users will enter the disk request queue and the initial head position.
- The system should display the order of servicing (seek sequence) and the total seek time for each algorithm, allowing clear comparative analysis.

Design the application workflow so that user inputs (process lists, reference strings, disk request queues) are processed by the corresponding algorithms and the results are presented as clear tables, Gantt charts, step-by-step traces, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate correctness. Optionally, students may add brief natural-language explanations of algorithm steps (no external LLM required).

Evaluate the application under suitable test cases and analyze correctness of algorithm results, usability of the interface, and clarity of visualizations. The completed system should also briefly address responsible-computing considerations including transparency of intermediate simulation steps, reproducibility of results, and educational ethics in presenting algorithm behavior.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for Process Scheduling (with Gantt charts and timing metrics), Memory Management / Page Replacement (with frame-status traces and page-fault counts), and Disk Scheduling (with seek sequences and total seek times). Accept user inputs as well as sample data. Produce comparative tables, charts, and step-by-step traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** Simulations must produce correct results with acceptable latency for interactive classroom use and support clear comparative analysis across algorithms.
- **Technical constraint:** Use Python + Streamlit. No database, no complex architecture, and no mandatory external LLM integration. Open-source tools only.
- **Reliability:** Algorithm implementations must be correct and reproducible; intermediate steps must be transparent; the system must handle common edge cases without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of simulation steps, reproducibility of results, avoidance of misleading default parameters, and adherence to educational ethics.
- **Evaluation & Submission:** Test all three modules with sample inputs and show correct results. Submit working Streamlit code, a short report containing sample outputs / screenshots, comparative analysis, and a GitHub repository link.

E. Student Work.

1. Problem Understanding and Formulation

1. OSLearn is an interactive web application developed to make Operating System concepts easier to understand.
2. The application focuses on **Process Scheduling, Memory Management, and Disk Scheduling**.
3. Users can enter their own inputs and observe how different OS algorithms work.
4. The system provides **Gantt charts, tables, frame status, page faults, and disk seek sequences**.
5. The main objective is to help students understand and compare the performance of different OS algorithms.

2. Application of Course Knowledge (CO2, CO3, CO4)

CO2 – Process Scheduling: Implemented FCFS, SJF, and Round Robin algorithms to understand CPU process execution.

CO3 – Memory Management: Implemented FIFO and LRU page replacement to understand page faults and memory frames.

CO4 – Disk Scheduling: Implemented FCFS, SSTF, and SCAN to understand disk request servicing and seek time.

Applied concepts such as **waiting time, turnaround time, page faults, and total seek time**.

Connected theoretical Operating System concepts with practical programming.

3. Solution / Design / Methodology

6. The application follows the workflow: **User Input → Algorithm → Calculation → Result → Visualization**.
7. Separate modules are provided for scheduling, memory management, and disk scheduling.
8. Python functions are used to implement each algorithm.
9. Scheduling results are displayed using **tables and Gantt charts**.
10. Memory management results show **frame status and page faults step by step**.
11. Disk scheduling results show the **seek sequence and total seek time**.

4. Use of Modern Tools

Python: Used to implement the Operating System algorithms.

Streamlit: Used to create the interactive web application.

Pandas: Used for displaying results in tables.

Matplotlib: Used for Gantt charts and visualizations.

VS Code: Used for coding, debugging, and testing.

GitHub: Used for storing and submitting the project code.

5. Results and Validation

12. Tested all three modules using different sample inputs.
13. Compared application results with manually calculated results.
14. Verified **waiting time and turnaround time** for scheduling algorithms.
15. Verified **page faults and frame changes** for page replacement algorithms.
16. Verified **seek sequence and total seek time** for disk scheduling algorithms.
17. The application produced correct and consistent results for the tested cases.

6. Analysis and Engineering Decision

18. **FCFS** is simple but can produce higher waiting time.
19. **SJF** can reduce average waiting time when shorter processes are selected first.
20. **Round Robin** provides fair CPU time to different processes.
21. **FIFO** is simple, while **LRU** uses recent page usage for better replacement decisions.
22. **SSTF** can reduce disk seek time, while **SCAN** provides more balanced disk servicing.
23. The suitable algorithm should be selected based on **performance, fairness, and system requirements**.

7. Broader Considerations

24. OSLearn helps students understand OS algorithms through practical visualization.
25. Step-by-step results improve **transparency and learning**.
26. The same input produces reproducible results.
27. The application can be extended with more algorithms in the future.
28. It can be useful for classroom demonstrations and self-learning.

8. Conclusion

29. OSLearn successfully combines important Operating System algorithms into one application.
30. The system provides an easy way to understand **CPU scheduling, page replacement, and disk scheduling**.
31. Visual results make algorithm comparison easier.
32. The project connects theoretical knowledge with practical implementation.
33. Python and Streamlit make the application simple and interactive.

9. Student Reflection

34. I gained practical knowledge of important Operating System algorithms.
35. I learned how to calculate **waiting time, turnaround time, page faults, and seek time**.
36. I improved my Python programming and problem-solving skills.
37. I learned how to create an interactive application using Streamlit.
38. This project helped me understand how OS concepts are applied in real systems.
- 39.

10. References

Abraham Silberschatz, Peter B. Galvin and Greg Gagne – **Operating System Concepts**.

William Stallings – **Operating Systems: Internals and Design Principles**.

Python Documentation.

Streamlit Documentation.

Pandas Documentation.

Matplotlib Documentation.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Scheduling, Paging, Disk)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Streamlit, Python)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Process Scheduling	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – Page Replacement	CO3	PO1, PO2, PO3	L3/L4	6
Application – Disk Scheduling	CO4	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated OSLearn	CO1–CO4	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Streamlit / Python)	CO1–CO4	PO5	L3	10
Validation of results	CO1–CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO4	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO4	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA04 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

